Agent题目

🥽0.Advisor顾问角色是如何设计和实现的

- Advisor顾问角色是我们AI Agent的核心能力之一，主要负责上下文记忆和知识库访问。
- 我们实现了两种主要的Advisor：PromptChatMemory用于维护对话历史，通过maxMessages参数控制记忆长度；RagAnswerAdvisor用于访问向量知识库，通过topK和filterExpression参数控制检索精度。
- 在实现上，我们继承了Spring AI的BaseAdvisor，重写了aroundCall方法来拦截对话请求，在请求前注入相关上下文信息，在响应后更新记忆状态。这种设计让AI Agent具备了持续学习和知识积累的能力，大大提升了对话的准确性和连贯性

🍑 1. 用户发起对话的全流程

- 第一步，**用户请求进入系统**：用户提交一条审核问题或者审核动作后，进入责任链的总入口 `RootNode`，在这里先把当前单子、证照类型、审核节点、风险等级这些基础上下文准备好，再把任务往后分发。
- 第二步，**组装本轮 Context Pack**：这一步通常由 advisor 和分析节点共同完成。`PromptChatMemoryAdvisor` 会先按 `sessionId` 或 `工单Id` 读取相关历史；同时系统不会把所有历史聊天原样塞回模型，而是只取和当前工单相关的内容，比如上一步查到的证照状态、缺失材料、工具执行结果，再把这些和任务上下文、RAG 证据候选一起组装成 `Context Pack`。这里我会做三件事：第一是隔离，按会话或工单分区；第二是预算，控制 window，必要时把旧消息做摘要；第三是筛选，只保留当前单子相关的有效信息。
- 第三步，**任务分析与场景识别**：进入 `Step1AnalyzerNode` 之后，系统会判断这次更偏基础信息校验，还是更偏高风险合规审查；同时把申请人信息、历史记录、黑名单、风险标签这些信息再整理一遍。
- 第四步，**执行**：进入 `Step2PrecisionExecutorNode`。基础信息型证照，一般会先调用 `queryLicense` 这类查询工具，去查证照台账、审核单状态、历史记录这些业务事实；高风险合规型场景则还会把历史审核记录、风险信息一起拉进来。然后 RAG 会按 `topK`、`filterExpression` 去召回当前证照类型对应的规范条款、合规条款和历史案例，把这些证据拼进 Prompt，再让模型输出结构化结果，比如 `decision`、`risk_level`、`missing_materials`、`violations`、`evidence`。
- 第五步，**结果兜底与后置校验**：模型给出结果后，不会直接当最终答案用，而是进入 `Step3QualitySupervisorNode`。这一层主要做两类事情：一类是证据质量检查，比如证据是否充分、结论和证据是否一致、有没有低置信度；另一类是确定性规则校验，比如必填项、格式、有效期、字段冲突。如果证据不足、证据冲突或者置信度不高，就不会强行给结论，而是拒答、降级或者转人工复核。
- 第六步，**结果回写与流程推进**：如果质量监督通过，才会进入业务回写阶段，比如 `writeBackResult`、`advanceFlow`，把审核结果真正落到业务系统并推进流程。这一步我会讲成“副作用节点”，因为它和查询型工具不一样，必须注意幂等和失败补偿，不能简单粗暴重试。

- 第七步，**Memory 分层写回**：这一轮执行结束后，memory分层写回。原始 user message 和 assistant message 最终落持久层；短期活跃窗口保留最近几轮对话和最新工具结果；如果窗口超过预算或者历史过程过长，就把旧内容压成结构化摘要。
- 第八步，**日志与可观测**：最后进入 `Step4LogExecutionSummaryNode`，把这轮审核用到的模型、命中的证据、调用过的工具、耗时、结果和错误码打成结构化日志，送到 ELK 做审计和后续巡检。也就是说，这个项目不是只有“模型答了什么”，还会记录“为什么这么答、调了哪些工具、耗时多少、在哪个节点失败”。

🧅 2. 用户询问之后，Memory机制的全流程

- 先确定这条请求属于哪个 `sessionId`，会带上证照类型、审核节点这些业务标识。因为后面所有 memory、任务状态、历史摘要的读取，都是靠这些 ID 做隔离和定位的。然后将原始user message落库
- 信息落库：用户的信息写入MySQL，写MySQL成功后再同步刷到Redis的活跃窗口，这一步由`MessageWindowChatMemory` 这种窗口策略负责裁剪
- 给大模型喂上下文，优先查 Redis 里的活跃会话窗口：最近几轮对话、最近工具结果、最近一版摘要，是下一轮最可能继续用到的内容，Redis不完整则去MySQL中 把最近几轮消息、结构化摘要、任务状态补回来，然后再回填到 Redis。之后memory和上下文、RAG检索结果一起组成Context Pack
- 模型开始执行后，如果过程中触发了 MCP 工具调用，比如查证照台账、查历史审核记录，这些工具结果不会只停留在模型上下文里，而是会先进当前轮次的临时状态区。短期上可以先进入 Redis，方便这一轮和下一轮快速读取；关键结果会被抽成结构化字段，比如“证照状态=已过期”“历史违规=1 次”，后面再更新到 MySQL 的任务状态或 summary 表，因为这些是审核事实，不能只让模型“看过就算了”。
- 当模型这一轮回复完成后，系统再把完整的 `assistant message` 落到 MySQL，并同步刷新 Redis 的最近窗口。这里不是每个 token 都写库，而是把这一轮完整回复作为一条 assistant message 持久化。然后系统判断这轮是否需要压缩：如果最近窗口已经超过阈值，比如 8 到 12 轮，或者 token 预算明显快超了，就会异步触发一次摘要任务。
- 摘要任务做成结构化抽取，比如 `taskState`、`confirmedFacts`、`riskFlags`、`missingMaterials`、`evidenceRefs`这些槽位。
- 摘要生成之后分层写回，一份写回 MySQL 的 summary 表，作为持久化的结构化摘要和任务状态；一份缓存到 Redis，供下一轮快速读取；如果这份摘要以后还适合按语义方式召回，再写一份到 PgVector

🍸 3. MCP和Function Calling的区别与联系

- Function Calling理解成：**大模型会“提议调用哪个函数、传什么参数”**。比如用户说“帮我查证照状态”，模型不会自己去查库，它只会先产出类似“调用 queryLicenseStatus，参数是证照号 xxx”的结构化结果，然后由你的后端程序真正执行这个函数，再把结果回给模型。所以它本质上是**模型发起工具调用的一种机制**，重点在“这一次该调谁、参数是什么”。
- MCP理解成：**一套把外部工具标准化接进大模型系统的协议**。也就是外部服务要按统一规则告诉模型系统：我这里有哪些工具、怎么调用、输入输出是什么、通过 stdio 还是 HTTP+SSE 通信。

- 核心的区别是：**Function Calling 讲的是“模型这次怎么调用一个工具”**，而 **MCP 讲的是“工具怎么以统一标准接进来”**。前者更像一次调用动作，后者更像一套接入规范。打个比方，Function Calling 像“我现在要按哪个按钮”，MCP 像“这个设备都按统一插座标准接进来”。所以 MCP 解决的是生态接入和解耦问题，Function Calling 解决的是模型侧的调用决策问题。
- 它们的联系：**MCP 负责把工具接进来，Function Calling 负责让模型在运行时决定要不要调、调哪个、传什么参数。**
- 电子证照项目里：**MCP 是工具接入层，Function Calling 是工具使用层。** 比如证照查询、流程回写、通知推送这些能力，我先按 MCP 协议封装成标准工具，通过 `defaultToolCallbacks` 挂到 ChatClient；当用户发起“审核这张证照”时，模型在推理过程中判断现在该不该调用“证照查询”或“状态回写”工具，这一步就是 Function Calling。

🌽 4. 怎么创建MCP Client

- 先创建 `McpSyncClient` 去连远程 MCP Server， `build()` 只是把 client 构出来； 另一个是 `initialize()` 才是真正按 MCP 生命周期去和服务端做初始化握手
- 把 `McpSyncClient` 包成 `SyncMcpToolCallbackProvider`，把远程工具统一转换成 Spring AI 的 `ToolCallback`
- 把 `ToolCallbackProvider` 注册进 `ChatClient`
- 执行时再由 `SyncMcpToolCallback` 通过底层 `McpSyncClient` 发起 MCP 调用。

🍟 5. RAG的流程是什么

- 流程上前半段是离线入库：文件先解析成文本，再切分成 chunk，做 embedding，存进向量库；后半段是在线问答：用户提问后，系统先做相似度检索，把命中的文档片段拼进 Prompt，再让模型基于这些证据回答

🍰 7. 解释一下A2A协议

- **Agent 和 Agent 之间通信、协作、委派任务的一套标准协议**。它解决的核心问题不是“模型怎么调工具”，而是“一个 Agent 怎么发现另一个 Agent、知道对方会什么、把任务交给对方、再把结果拿回来”。
- **Agent 世界里的 RPC / 微服务调用协议**。比如主 Agent 负责统筹全局，但它发现“财务测算”“商品目录查询”“风控审批”这些任务更适合交给专业 Agent 去做，这时候就可以通过 A2A 去调用远端 Agent。
- 它和 MCP 的区别：**MCP 解决的是 Agent 怎么接工具，A2A 解决的是 Agent 怎么接另一个 Agent。** Google 官方也明确说了，A2A 是对 MCP 的补充关系，不是替代关系。MCP 更像“给当前 Agent 提供工具和上下文”，比如查数据库、调流程、发通知；A2A 更像“把一个完整的专业 Agent 当成远程能力来协作”，比如我的审核 Agent 不直接自己做所有事，而是把某一类任务委托给另一个独立的合规 Agent 或搜索 Agent。

🍹 8. Skill了解吗？底层是如何实现的？

- Skill用来告诉 ChatGPT 某个任务“具体应该怎么做”，里面可以带指令、示例，甚至代码。装好之后，ChatGPT 可以在合适的时候自动调用一个或多个 Skill。一个 Skill 通常会包含名字和描述、`SKILL.md` 里的工作流说明，以及任务依赖的资源。
- 从形态上看，Skill 本质是一个**目录或 zip 包**，核心入口是 `SKILL.md`。这个文件里一部分是元信息，比如 name 和 description；另一部分是具体行为说明。
- 除了 `SKILL.md`，Skill 还可以带脚本、参考资料、模板这些附加文件。也就是说，Skill 不是模型参数里“学会了什么”，而更像是**运行时挂载的一组外部任务定义**。Skill 的价值是把一个重复任务的“做法”保存下来，让 ChatGPT 在合适场景下稳定复用，而不是每次从零开始。
- Skill它本质上是“路由 + 懒加载 + 可选执行”的组合。
- **第一步**，系统先常驻看到的不是整个 Skill 内容，而只是很轻量的元信息，比如这个 Skill 叫什么、适合什么任务。模型收到用户请求后，先做一次匹配判断：这件事要不要触发某个 Skill。**第二步**，一旦命中，才去加载这个 Skill 的核心说明，也就是 `SKILL.md` 的主体，让模型知道这次任务应该遵循什么流程。**第三步**，如果当前步骤真的需要更细的资源，比如脚本、参考文档、模板，再继续按需读取或执行。
- 这个过程本质上就是把 Skill 当成一个**外部可导航的文件系统资源包**，而不是一上来把所有内容全塞进上下文里。

🍉 10. 上下文窗口满了如何去压缩?

- 压缩的是历史对话里的任务状态、关键事实、证据引用、待办动作

1. 压缩前置条件

- 轮次超阈值：当前会话最近窗口已经超过 8 到 12 轮
- token 预算超阈值：把最近几轮、任务上下文、RAG 证据都算进去后，已经接近模型可用上下文上限

2. 确定压缩哪些内容

- 不压缩：system prompt、当前证照类型、最新工具结果
- 压缩：已经被后续覆盖的讨论过程、已经完成的分析过程、较早的 user / assistant 多轮往返
- 长期检索区：历史相似案例、典型拒答原因

3. 新摘要和旧摘要做Merge操作

- 同一个事实不重复保存，只保留最新版。taskState 后写覆盖前写
- pendingActions 动态更新，已经完成的待办从 pending 里移除，新待办补进去。

4. 压缩成固定的Json格式

```
{
  "workOrderId": "WO-20250101-001",
  "licenseType": "营业执照",
  "currentNode": "初审",
  "confirmedFacts": [
    "证照号已核验",
    "企业名称与申请信息一致",
    "证照有效期至 2026-12-31"
  ],
  "riskFlags": [
    "历史存在1次违规记录"
  ],
  "missingMaterials": [
    "未上传年检证明"
  ],
  "completedSteps": [
    "基础字段校验",
    "证照台账查询",
    "历史记录查询"
  ],
  "pendingActions": [
    "补充年检证明",
    "做合规条款校验"
  ],
  "evidenceRefs": [
    "tool:queryLicense",
    "tool:queryHistory",
    "rag:spec-营业执照-年检条款"
  ],
  "unconfirmedHypotheses": [
    "疑似存在经营范围超限，待进一步核验"
  ],
  "lastDecision": "证据不足，暂不通过，待补件"
}
```

5. 压缩之后写回

🍿11.如何筛选有效信息和垃圾信息？

- 第一是相关性，只保留和当前工单、当前证照类型、当前审核节点相关的内容
- 第二是新颖性，重复表达同一个事实要去重，避免污染上下文
- 第三是可信度，工具结果、数据库返回、用户明确给出的约束，优先级高于模型自己推断出来的话
- 第四是时效性，旧状态、过期计划、已经被覆盖的参数要及时淘汰。比如上一轮说“处理中”，这一轮工具已经返回“已驳回”，那旧状态就不能继续保留在活跃上下文里。

Memory存redis或者MySQL

- **Redis 这一层我会放短期热点上下文**，比如最近 5 到 10 轮对话、当前压缩后的摘要、最近一次工具结果、当前工单的临时上下文。因为这些数据每次模型调用前都要读、每轮执行后都要写，读写频率高、生命周期短，放 Redis 更合适。MessageWindowChatMemory最大消息窗口为20
- **MySQL 这一层我会放任务记忆和结构化摘要。** 比如工单当前节点、风险标签、缺失材料、关键事实快照、已完成步骤、待办步骤、人工复核标记、摘要版本、冲突覆盖记录。因为审核系统很多信息后面要追溯、要恢复、要审计，尤其结果回写之前还要做后置校验，所以一定要有一层持久化存储，把状态真正落下来。

🧊 11. 解释一下上下文工程中Agent的Memory是怎么实现的

- 实现上分成两部分：**写入机制**和**读取机制**。写入时，系统不会把所有内容原样存进去，而是先做筛选，只保留有价值的信息，比如用户长期偏好、当前任务状态、重要事实、工具返回结果；闲聊、重复话、过期状态一般会丢掉。读取时，也不是把全部记忆一次性塞回模型，而是根据当前问题去召回最相关的那部分，再和当前输入一起注入上下文。
- 业界一般会把 Memory 分成几类。**短期记忆**，也叫 working memory，主要保存当前会话最近几轮内容，常见做法是 sliding window 或摘要压缩；**长期记忆**，保存跨轮、跨任务仍然有价值的信息，比如用户偏好、历史任务结论、稳定事实，一般会落数据库、KV、向量库；还有一种是**任务记忆**，更像状态机，记录当前任务做到哪一步、上一步工具返回了什么、下一步该干什么。 Agent 三类混合用。

- 我项目里 Agent 的 Memory 是通过 Spring AI 的 `ChatMemory + PromptChatMemoryAdvisor` 实现的。 `ChatMemory` 负责存多轮消息，Advisor 在每次调用前按会话或工单读取相关历史注入 Prompt，在调用后再把本轮结果写回；同时我不会把 Memory 当全部上下文，而是把它和任务上下文、RAG 证据一起组成 Context Pack，再配合窗口控制、按工单隔离和优先级策略
- 第一是**隔离**，Memory 一定按 `sessionId` 或 `工单Id` 分区存，不能不同用户、不同单据共用一份记忆；第二是**预算**，不能无限追加历史，要控制 window，必要时把旧消息做摘要；第三是**筛选**，只保留和当前单子相关的内容，比如上一步查到的证照状态、缺失材料、工具执行结果，闲聊和过期状态要丢掉。

🍷 12. Agent编排是什么意思？不同证照Flow的具体例子

- **Agent 编排**，是指把证照审核这件事拆成了一套**可配置、可扩展的执行流程**。项目里不是把审核逻辑写死在一个大函数里，而是先把 Agent、模型、工具、Advisor、提示词、知识库这些配置加载出来，组装成可执行的 ChatClient；运行时再根据 `agentId` 找到对应能力，结合 system prompt、MCP 工具、RAG 和 Memory 去完成当前这轮审核。这样不同证照类型、不同审核节点，都可以通过改 Flow 和配置扩展，不需要每来一个场景就改一版代码。

- 基础信息校验型证照：这类证照的核心目标是先把“这张证照本身对不对”查清楚，所以 Flow 会比较偏确定性校验。用户发起审核后，系统先根据 `agentId` 命中对应 ChatClient，把当前单子的任务上下文先组起来，里面包括证照类型、审核节点、风险等级、申请人信息这些字段；然后模型第一步通常会先调 `queryLicense` 这种查询工具，去查证照台账、审核单状态、历史记录这些业务事实。事实拿回来之后，再让 RAG 去召回这一类证照对应的规范条款和历史案例，最后模型输出结构化结果，比如 `decision`、`risk_level`、`missing_materials`、`violations`、`evidence`。输出完不是直接落库，还要再做一轮后置校验，比如必填项、格式、有效期、字段冲突这些，没问题才走 `writeBackResult` 和 `advanceFlow`，把审核结果回写并推进流程。
- 高风险合规型证照：这类证照更关心“有没有合规风险、证据够不够”。它的公共骨架和前面一样，也是先组 Context Pack，但任务上下文里会更强调风险等级、历史记录、黑名单这些信息；模型调工具时，也不只是查证照当前状态，还会把历史审核记录、风险信息一起拉进来。然后 RAG 检索阶段会更重，因为它要从证照规范、合规条款、历史案例里召回更贴近当前场景的证据片段。接着模型生成审核意见时，必须把 `evidence` 一起带出来，不能只给结论不给依据；如果后面发现证据不足、证据冲突，或者模型置信度不高，就不会强行给通过/驳回，而是直接拒答或者转人工复核。

🥖 13. 简历第四点：安全与可观测

- 可观测是**从 Agent 执行链路里主动打结构化日志**。因为项目里真正重要的信息，不只是“这个 HTTP 请求来了”，而是**这次 Agent 用了哪个模型、调了哪些工具、耗时多少、有没有报错、最后结果是什么**。所以我会在 `ChatClient` 这一层挂日志类 Advisor，比如 `SimpleLoggerAdvisor`，每次 `chatClient.prompt(...).call()` 或工具调用前后，把关键字段写成 JSON 日志。
- 日志出来之后，下一步是**采集和入 ELK**。日志顾问写 JSON → Filebeat / Logstash 收集 → Elasticsearch 存储 → Kibana 查询和看图。
- 定时巡检 Agent：由 **Spring TaskScheduler** 按 Cron 定时触发。它会周期性地去看 ELK 里的日志指标和业务侧的积压指标，比如：某个审核 Agent 在一段时间内失败率是不是突然升高。某个工具调用是不是持续超时待审核单是不是长时间堆积。某个节点是不是出现“处理中很多，但完成很少”的积压现象

14. 🥒巡检 Agent 来做积压监控，怎么定义“积压”？

- 失败率只是一个很粗的结果指标，真正能提前发现问题的是**过程指标**。我会按阶段看。比如收到审核请求很多，但 `TOOL_DONE` 很少，说明可能卡在工具侧；`VALIDATING` 很多但 `WRITEBACK_PENDING` 很少，说明可能是后置校验或规则冲突过多；`WRITEBACK_PENDING` 堆积，则可能是数据库写回或流程服务卡住了
- 所以巡检 Agent 不只是看成功率，而是看一组指标：各阶段数量漏斗、各阶段平均耗时和 P95、工具超时率、低证据拒答率的突增、无证据输出率、回写失败率、通知延迟、待审核单存量。然后结合业务阈值和历史基线做判断。比如某类证照平时低证据拒答率是 8%，今天突然到 35%，即使服务没挂，我也会报警，因为这通常意味着知识库版本、解析链路或者过滤条件出了问题。

- 巡检触发后，我不会让它直接“自动修复一切”，但至少可以自动给出定位建议，比如是模型时延上升、RAG 无命中增多、某个工具超时，还是某个节点的流程状态长时间不推进。这样运维和开发接手时不是从零查

15. harness and SDD

harness

- 给 Agent 设计一套能被约束、被观察、被审查的运行环境
- 一是 **feedforward**，也就是事前约束，比如 AGENTS.md、架构原则、技能文件、脚本模板、目录约定
- 二是 **feedback**，也就是事后校验，比如测试、lint、类型检查、日志、代码评审 agent、运行指标
- 三是 **可观测与可恢复**，让 agent 看得见 UI、日志、指标，并能在失败后继续迭代。

SDD

把“要做什么、为什么这么做、什么算完成”写成结构化规范，再让实现、测试、文档围着这份规范展开，通常包括：

- 需求：系统要实现什么
- 约束：性能、安全、兼容性、合规、技术边界
- 验收标准：什么叫完成
- 设计意图：为什么选这个方案
- 任务拆解：实现时怎么分步推进

16. 具体说一下策略模式是怎么实现的，Agent自己解析具体的策略吗

- 第一步，系统先根据业务条件，比如 **证照类型、审核节点、风险等级**，先命中对应的 Flow 或对应的执行策略。
- 基础信息校验型，适合营业执照、许可证这类‘先确认证照本身是否有效’的场景；第二类是高风险合规型，适合更看重合规条款、风险证据的场景。”
- 基础信息校验型：证照基础字段是否齐全(证照号、主题名称)，有效性检查。
- 高风险合规型：还会把历史审核记录、风险信息一起拉进来；然后让 RAG 去检索对应的证照规范、合规条款、历史案例，最后模型输出时必须带 `evidence`，不能只说“有风险”却不给依据。要是出现证据不足、证据冲突、置信度不高，就不强行给结论，而是直接拒答或者转人工复核
- 第二步，策略选定之后，再进入责任链执行，也就是 `RootNode -> Step1AnalyzerNode -> Step2PrecisionExecutorNode -> Step3QualitySupervisorNode -> Step4LogExecutionSummaryNode` 这一套。这里每个节点都只负责一段固定职责，所以策略模式和责任链模式是配合使用的：**策略决定走哪套大方案，责任链负责这套方案内部怎么一步步跑。**

责任链

- `RootNode` 是总入口，负责把这次审核任务接进来，把 当前单子、证照类型、审核节点、风险等级这些基础上下文准备好，再把任务往后分发。
- `Step1AnalyzerNode` 是任务分析节点，主要做场景识别和上下文整理，比如这次更偏基础信息校验，还是更偏高风险合规；同时把申请人信息、历史记录、黑名单、风险标签这些组装成 Context Pack。
- `Step2PrecisionExecutorNode` 是精准执行节点，这一步才是模型真正开始干活的地方。比如基础信息型证照，我会先调 `queryLicense` 查证照台账、审核单状态、历史记录，再让 RAG 去召回对应证照的规范条款和历史案例，最后让模型输出结构化结果；高风险合规型则会更重视风险信息、黑名单和证据充分性。
- `Step3QualitySupervisorNode` 是质量监督节点，它不是重新做一遍审核，而是做结果兜底，检查证据是否充分、结论和证据是否一致、有没有低置信度，同时做后置确定性校验，比如必填项、格式、有效期、字段冲突；证据不足就拒答或者转人工。
- `Step4LogExecutionSummaryNode` 是执行总结节点，把这轮审核用到的模型、命中的证据、调用过的工具、耗时、结果和错误码打成结构化日志，方便审计、ELK 检索和后续巡检。

17. 技术选型(框架、向量数据库)

springAI

- java技术栈、如果后面这个系统往更重的工作流 Agent 演进，比如需要更强的状态图编排、跨节点恢复、人工审批插点更复杂，我会认真考虑 LangGraph，甚至会觉得它在那种阶段更合适。

向量数据库

- MongoDB具有ANN索引调优，其次就是**hybrid search**，也就是把向量搜索和全文搜索一起用。支持把向量结果和全文结果做融合，还可以做权重调整、语义 boosting、rank fusion 之类的策略。但是我的项目场景里主要是按照`filterExpression` 去做召回，也就是先把候选集收敛到当前证照类型、审核节点、风险条件，再做相似度检索。
- 其次就是**高级预过滤**。MongoDB 支持在向量搜索前，对布尔、日期、数值、字符串、UUID 等字段做预过滤；进一步还支持带 analyzed text 能力的 lexical prefilters，比如 fuzzy、phrase、wildcard、地理位置等高级过滤。这个在知识库很大、候选范围很广时很有用。
- 在目前的文档数量情况下，MongoDB 这些真正有技术含量、也最能体现平台价值的能力——比如 ANN 参数调优、hybrid search、高级预过滤、量化优化——大概率暂时都用不上。